

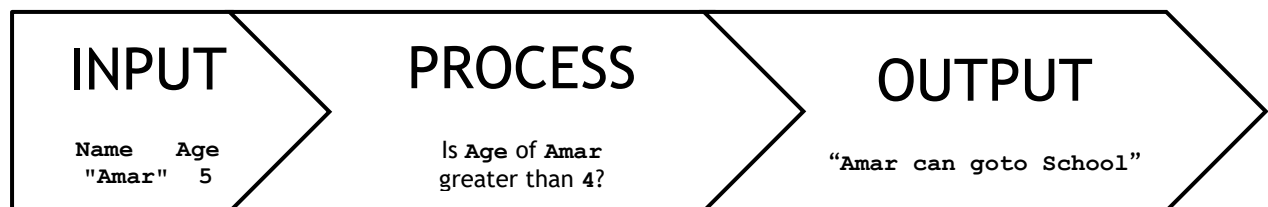
Python Operators I

Operators are special symbols which represent computation. They are applied on operand(s), which can be values or variables. Some operators can behave differently on different data types. Operators when applied on operands form an expression. Operators are categorized as Arithmetic, Relational, Logical and Assignment. Value and variables when used with operator are known as operands.

Arithmetic Operators

Operator	Meaning	Expression	Result
+	Addition	10 + 20	30
-	Subtraction	30 - 10	20
*	Multiplication	30 * 100	300
/	Division	30 / 10	20.0
		1 / 2	0.5
//	Integer Division	25 // 10	2
		1 // 2	0
%	Remainder	25 % 10	5
**	Raised to power	3 ** 2	9

Python Input and Output



Python Output Using print() function

We use the `print()` function to output data to the standard output device (screen). We can also output data to a file. An example is given below.

```
a = "Hello World!"  
print(a)
```

The output of the above code will be:

```
Hello World!
```

Example Code	Sample Output
--------------	---------------

Python Operators II

Comparison operators

Operator	Meaning	Expression	Result
>	Greater Than	20 > 10	True
		15 > 25	False
<	Less Than	20 < 45	True
		20 < 10	False
==	Equal To	5 == 5	True
		5 == 6	False
!=	Not Equal to	67 != 45	True
		35 != 35	False
>=	Greater than or Equal to	45 >= 45	True
		23 >= 34	False
<=	Less than or equal to	13 <= 24	True
		13 <= 12	False

Comparison operators are used to compare values. It either returns True or False according to the condition.

Logical operators

Operator	Meaning	Expression	Result
And	And operator	True and True	True
		True and False	False
Or	Or operator	True or False	True
		False or False	False
Not	Not Operator	not False	True
		not True	False

Logical operators are the and, or, not operators.

Chapter 4 - More About Lists and Tuples

Introduction to Lists

As studied in the previous section, List is a sequence of values of any type. Values in the list are called elements / items. List is enclosed in square brackets.

Example:

```
a = [1,2.3,"Hello"]
```

List is one of the most frequently used and very versatile data type used in Python. A number of operations can be performed on the lists, which we will study as we go forward.

How to create a list ?

In Python programming, a list is created by placing all the items (elements) inside a square bracket [], separated by commas.

It can have any number of items and they may be of different types (integer, float, string etc.).

Example:

```
#empty list
empty_list = []

#list of integers
age = [15,12,18]

#list with mixed data types
student_height_weight = ["Ansh", 5.7, 60]
```

Note: A list can also have another list as an item. This is called nested lists.

```
# nested list
student_marks = ["Aditya", "10-A", [ "english",75]]
```

How to access elements of a list ?

A list is made up of various elements which need to be individually accessed on the basis of the application it is used for. There are two ways to access an individual element of a list:

- 1) List Index
- 2) Negative Indexing



There come situations in real life when we need to make some decisions and based on these decisions, we need to decide what should we do next. Similar situations arise in programming also where we need to make some decisions and based on these decisions we will execute the next block of code.

Decision making statements in programming languages decide the direction of flow of program execution. Decision making statements available in Python are:

- if statement
- if..else statements
- if-elif ladder